

26F Stat TA Session W9

0. Preparation for TA Session

- Download (and import) the sample code and attached datasets: `babies` & `restaurants` [click here](#)
 - the `babies` dataset includes the babies born in 1960-1967 in the San Francisco East Bay area
 - the `restaurant` dataset is credited to the TA of last year!!!

1. Logical Expressions

- This week, we will introduce you to the world of data wrangling (limbo). But before we run into the trivial work, let's first take a look at the useful logical expressions

Comparison Operators `>` , `<` , `=` , `!`

- You should be familiar with these operators (except for the last one `!` (negative))

```
> Loading R code...
```

Logical Combination `&` , `&&` , `|` , `||`

- You can also combine multiple statements to form a more complex logical statement

AND `&` (element wise), `&&` (first element only)

```
> Loading R code...
```

OR `|` (element wise), `||` (first element only)

```
> Loading R code...
```

Q. How to choose between element wise (`&` , `|`) and first element only (`&&` , `||`)?

💡 Ans. Choose first element only one if

- somehow you only want to compare the first element in the vectors
- it's inside an `if` statement or a `for` loop (there should be only a single logical condition for control flows)

```
> Loading R code...
```

- `&` and `|` are also useful filters

```
> Loading R code...
```

`%in%`

- Recall that we have encountered this operator in W6 exercises, it is used to check if an object is inside a vector or a list

```
> Loading R code...
```

Use `()` for Clearer Expressions

- Coding Style matters! When the logical expression is too complicated or too long, add `()` to improve the readability! (Also save your time from debug)
- `()` can also help you understand the hierarchy (order) of operators (just like `()` in math calculations)

```
> Loading R code...
```

2. Data Wrangling with `dplyr`

- Just as plotting with `ggplot2`, we (hopefully) have a useful and powerful tool for data wrangling, named `dplyr`

`dplyr` and `tidyverse`

- `dplyr` is grammar of data manipulation in `R`
 - Core functions: `filter()`, `select()`, `mutate()`, `arrange()`, `summarize()`, `group_by()`
 - Another useful tool: `%>%`
- `tidyverse` is a collection of packages including `ggplot2`, `dplyr`, `tidyr`, `readr`, etc.
 - You can also load `tidyverse` in `RStudio` without further specifying which exact packages you are using (e.g., `dplyr`)

Pipeline `%>%`

- The pipeline operator `%>%` in `dplyr` is used to chain multiple operators on data together

```
> Loading R code...
```

What's in the Dataset?

- Now let's take a look at the `babies` dataset
 - `case` : id number
 - `bwt` : birthweight
 - `age` : mother's age in years
 - `height` / `weight` : mother's height/ weight in inches/ pounds

```
> Loading R code...
```

`filter()`

- Suppose we want to filter the mothers with height 62-64 inches (~ 157.5-162.5 cm), we use `%>%` and `filter()`

```
> Loading R code...
```

- Another example can be like looking for smoking mom and babies with birth weight > 120 (there are 171 such examples...)

```
> Loading R code...
```

- You may have noticed that there are `NA` values (missing values) in the dataset
- By default, `filter()` does not keep rows with `NA`

`select()`

- There are too many columns (variables) in the dataset. Suppose now we only want to focus on columns `bwt` and `age`, we can use `select()` to select and keep these columns

```
> Loading R code...
```

- We may also delete certain columns

```
> Loading R code...
```

Create New Columns by Existing Variables `mutate()`

- In most cases, the variable of our interests may not be recorded in the original dataset (e.g., the BMI of moms). Then, we have to compute the numbers and create a column ourselves
 - The BMI is left as exercise (hint: be aware of the unit used!). In the example, we show how to create a column for height in cm

```
> Loading R code...
```

- `mutate()` is also very powerful for condition grouping!
 - If the grouping condition is binary, we can use together with `ifelse()`
 - If groups ≥ 3 , use together with `case_when()`

```
> Loading R code...
```

Sort the Data `arrange()`

- Suppose we want the rows to be in descending order according to the mother's height
 - By default, the order is ascending

```
> Loading R code...
```

Caveats

- These `dplyr` functions do not change the original dataset (unless you assign the output back to the variable itself)
- Always use pipeline to improve readability and keep your code maintainable

```
> Loading R code...
```

Exercises

 [W9 Exercises](#)



由Print Notion導出